



Understanding Concurrency Bugs

Concurrency has come of age with the wide use of multi-core processors. In this article, let us explore the importance of writing correct concurrent code.

Multi-core processors have really become mainstream these days. It is common to see mobile phone processors with dual-cores, with some new models even having quad-cores. Almost all computers (laptops, servers, etc) have multiple cores. With the wide use of multi-core processors, it has become more important than ever before to write concurrent code to exploit the power of these processors.

In the past, lots of multi-threaded code was written but for single-core processors. Concurrent code was written mainly for running tasks in the background, to provide responsive user interfaces, etc. But when we start using these applications in systems with multiple cores, the applications become really concurrent and concurrency bugs start showing up.

Writing correct concurrent code is not easy. With every thing else being equal, concurrent code can be expected to have more problems than sequential (deterministic) code. Why? Sequential programs are influenced by input, the systems environment and user interaction. In addition to these factors, concurrent programs are influenced by the ordering of events (such as scheduling, which is non-deterministic). Testing concurrent programs is also difficult. There are two main reasons for this—limited observability and controllability. The tester cannot observe important details of program execution, like the interleaving of threads. The tester also cannot easily reproduce the problems, limiting the controllability. Experts Herb Sutter and James Larus put it succinctly "...humans are quickly overwhelmed by concurrency and find it much more difficult to reason about concurrent than sequential code. Even careful people miss possible inter-leavings..."

When I wrote concurrent programs, I got exposed to different kinds of concurrency problems. I always wondered why no one told me about the fundamental kinds of concurrency problems that one ought to be aware of. So, I created a quick and simple classification of concurrency bugs, which has only three categories of problems that you need to remember: determinism-related, safety-related, and 'liveness'-related. Well-

known definitions of these three properties are:

- **Determinism:** Ensure that, for a given set of inputs, the output values of a program are the same for any execution schedule.
- **Safety:** Ensure that nothing bad happens.
- **Liveness:** Ensure that something good eventually happens.

Determinism-related bugs

Data races (also known as race conditions) are perhaps the best known bugs related to determinism.

Typically, when we talk about a data race, we discuss the low-level data race when two or more concurrent threads access a shared variable and when at least one access is a write; and the threads use no explicit mechanism (such as a mutex) to prevent the access from being simultaneous. However, a data race could also be high-level when a set of shared variables need to be accessed or modified together atomically.

There are many other kinds of determinism bugs as well. For instance, when the code depends on thread scheduling, it can cause subtle bugs. I remember cases in which programmers had used sleep calls instead of using mutex or the wait/notify pattern for safe access to shared variables. In such cases, when the programmers try to use the application in their machines, it may work fine, but in a testing or production environment, the bug may get exposed, as in the following real-world incident.

In August 14, 2003, millions of people lost electric power in northern USA and Canada. There were several factors contributing to the blackout, and the official report indicated a problem in a C++ alarm monitoring software. There was a data race caused because of artificially introduced delays in the code. Because of this race condition, the alarm event handler went into an infinite loop and failed to raise an alarm. This eventually led to a power blackout.

Safety-related bugs

A well-known safety-related concurrency bug is 'missing